

Guía de Estudio

Gradient-Based Program Synthesis with
Neurally Interpreted Languages (NLI)

Macfarlane, Bonnet, van Hoof & Lelis — ICLR 2026

Guía pedagógica preparada con Claude, pensada para
una primera introducción a la síntesis de programas
basada en gradientes

Contents

Cómo usar esta guía	2
1 Panorama general: el problema que resuelve el paper	3
2 Introducción	4
3 Planteamiento del problema (Problem Statement)	5
4 Inferencia secuencial discreta: el corazón de NLI	7
4.1 Objetivo de entrenamiento (<i>Training Objective</i>)	7
4.2 El inductor: aprendiendo una representación discreta del programa	9
4.3 El intérprete: ejecución neuronal recurrente de programas	11
4.4 Búsqueda de programas neuronales en tiempo de prueba	13
5 Experimentos	14
5.1 Generalización composicional: los benchmarks Shift-L, Shift-P y Comp-I	14
5.2 Ablaciones: ¿qué componente es realmente responsable del éxito?	16
5.3 El rol del Gumbel-Softmax	16
5.4 Escalando la búsqueda en tiempo de prueba	17
5.5 DeepCoder: comparación con métodos neuro-simbólicos	17
6 Trabajo relacionado	18
7 Conclusión, limitaciones y trabajo futuro	19
8 Glosario de términos clave	20
9 Autoevaluación final	22

Cómo usar esta guía

Este documento no reemplaza al paper: es un **andamiaje** para leerlo. La idea es que avances sección por sección, en el mismo orden en que están en el original, pero con paradas frecuentes para que la intuición llegue antes que la fórmula, y para que tengas oportunidad de pensar por tu cuenta antes de que te dé la respuesta.

Un consejo de lectura: no intentes entender cada ecuación en la primera pasada. La primera vez, lee solo las cajas de **intuición** y de **perspectiva** de cada sección para hacerte una idea del mapa completo. En una segunda pasada, vuelve sobre las cajas de **ecuación desglosada** con lápiz y papel al lado.

A lo largo del texto usaremos cinco tipos de cajas, siempre con el mismo color y el mismo propósito:

Intuición: ejemplo

Así se verá una explicación intuitiva: usaremos analogías y lenguaje cotidiano antes de mostrar cualquier fórmula.

Desglosando la ecuación: ejemplo

Así desglosaremos una ecuación, símbolo por símbolo, explicando qué representa cada término y por qué está ahí.

¿Por qué importa esto?

Así conectaremos cada concepto con aplicaciones reales o con el resto del campo de la IA, para que entiendas el "para qué".

Para pensar

Así te haremos preguntas abiertas para que reflexiones antes de seguir leyendo, o para autoevaluar tu comprensión.

Ejemplo de término

Así definiremos formalmente los términos técnicos nuevos la primera vez que aparezcan (por ejemplo: DSL, Gumbel-Softmax, latent space, etc.).

1 Panorama general: el problema que resuelve el paper

Antes de abrir el paper, conviene entender el problema de fondo, porque casi todas las decisiones de diseño de NLI son respuestas a este dilema.

Síntesis de programas es el problema de, dado un comportamiento deseado (por ejemplo, unos pocos ejemplos de entrada-salida), encontrar *un programa* que lo reproduzca. Es distinto de *aprender una función* al estilo de una red neuronal clásica: aquí el objetivo es recuperar algo con estructura de programa —composicional, reutilizable, interpretable— y no solo una aproximación numérica.

Históricamente, este problema se ha atacado desde dos escuelas:

- **Síntesis simbólica.** Se define un *Lenguaje Específico de Dominio* (DSL, ver definición más abajo) con un conjunto fijo de operaciones primitivas, y se *busca* (por fuerza bruta, heurísticas, o SAT-solvers) la combinación de esas operaciones que explica los ejemplos. Ventaja: si encuentra un programa, ese programa es exacto, interpretable, y generaliza de forma composicional casi por definición. Desventaja: diseñar el DSL correcto para cada dominio es trabajo humano costoso, y buscar dentro de él es combinatoriamente explosivo.
- **Síntesis neuronal.** Se entrena una red (por ejemplo, un transformer) para mapear directamente ejemplos de entrada-salida a una predicción, sin pasar por ningún programa explícito. Ventaja: aprende de datos, sin diseño manual de primitivas. Desventaja: lo que aprende queda *enredado* en los pesos de la red. Cuando el problema en test requiere *recombinar* ideas ya vistas de una forma nueva (generalización composicional), la red neuronal típica falla, incluso si en principio “ya sabía” cada pieza por separado.

Intuición: el dilema en una frase

Lo simbólico generaliza bien pero cuesta mucho construirlo a mano; lo neuronal se construye solo (aprende de datos) pero generaliza mal cuando hay que recombinar piezas de maneras nuevas. NLI intenta quedarse con lo bueno de ambos mundos: *aprender* un lenguaje simbólico, en vez de diseñarlo a mano.

La propuesta central del paper es una arquitectura llamada **Neural Language Interpreter (NLI)**. En una frase: NLI aprende, únicamente a partir de ejemplos de entrada-salida, (1) un *vocabulario* propio de “instrucciones” discretas (su propio DSL, pero aprendido y no diseñado a mano) y (2) un *intérprete* neuronal diferenciable capaz de ejecutar secuencias de esas instrucciones. Como todo el sistema es diferenciable, en el momento de resolver un problema nuevo (*test time*) se puede usar descenso de gradiente para *buscar* el mejor programa, algo que un DSL simbólico tradicional no permite (ahí solo se puede enumerar o usar heurísticas discretas).

El flujo de trabajo de NLI en tiempo de inferencia tiene tres actores, que iremos viendo en detalle en la Sección 4:

1. El **inductor** (el encoder) mira los ejemplos disponibles y propone una primera secuencia de tokens: un “primer borrador” de programa.
2. La **búsqueda por gradiente** refina ese borrador, moviéndose en una versión continua y relajada del espacio de programas para encontrar uno que explique mejor los ejemplos.
3. El **intérprete** (el decoder) ejecuta el programa encontrado, token a token, sobre una entrada nueva, para producir la predicción final.

¿Por qué importa esto?

Este patrón —proponer con una red, refinar con optimización, ejecutar con otra red— no es exclusivo de la síntesis de programas. Aparece en difusión (proponer ruido, refinar con gradiente hacia la imagen), en planificación robótica (proponer una trayectoria, refinarla), y en general en cualquier problema donde “adivinar y luego pulir con gradientes” es más barato que resolver el problema exacto de una sola pasada. Entender NLI te da una plantilla mental reutilizable para otros problemas de IA.

Para pensar

Antes de seguir: ¿por qué crees que es *necesario* que el intérprete sea diferenciable para que la búsqueda por gradiente en el paso 2 sea posible? ¿Qué pasaría si el intérprete fuera, por ejemplo, un intérprete de Python real, no una red neuronal?

2 Introducción

Para entender por qué NLI está diseñado como está, hay que conocer a su “padre” directo: la arquitectura **Latent Program Network (LPN)** (Macfarlane & Bonnet, 2025), publicada por (en parte) los mismos autores.

Latent Adaptation Network (LAN)

Familia de arquitecturas encoder-decoder que aprenden un *espacio latente* para representar el espacio de posibles programas/comportamientos. Su característica definitoria es que, en tiempo de prueba, ese espacio latente se *busca* (se optimiza) para adaptarse a la tarea concreta que se está resolviendo, en vez de solo hacer una pasada directa hacia adelante (*forward pass*) con parámetros fijos.

LPN es la primera instancia de esta familia: usa un espacio latente *continuo* (un simple vector de números reales) para representar el programa, y condiciona al decoder con ese vector en una única pasada. Tiene virtudes reales: es resistente al sobreajuste (porque el vector latente es una representación muy comprimida) y permite adaptación rápida en dominios como reparación de programas. Pero el propio paper de LPN mostró una limitación importante: **generaliza mal composicionalmente**. Si el vector latente representa “todo el programa” de una vez, no hay ninguna presión para que ese vector se organice en piezas reutilizables y recombinables; el decoder puede aprender un mapeo continuo arbitrario que funciona en distribución pero no tiene ninguna estructura interna que se pueda “desarmar y rearmar” para tareas nuevas.

NLI es también una LAN —también busca en un espacio latente en tiempo de prueba— pero cambia la pieza clave: en vez de un vector continuo único, usa una **secuencia de tokens discretos**, como si fuera código fuente en un lenguaje inventado por la propia red. La apuesta de los autores es que la discreción, sumada a una ejecución *recurrente* (token a token, en vez de todo el programa de una vez), es justo lo que faltaba para inducir estructura composicional: un token puede significar “desplazar la lista una posición a la izquierda”, y ese mismo token puede reaparecer, reutilizado, dentro de programas completamente distintos.

Intuición: LPN vs. NLI en una imagen mental

Pensá en LPN como si describiera un programa completo con *una sola palabra muy larga y muy específica* (un vector continuo ad-hoc para cada tarea). NLI, en cambio, describe el mismo programa con una *oración*, hecha de palabras (tokens) de un vocabulario pequeño y compartido entre todas las tareas. Es mucho más fácil reutilizar palabras sueltas en oraciones nuevas que reutilizar una palabra larguísima e irrepetible.

Otra diferencia importante, consecuencia directa de la anterior: como el intérprete de NLI consume la secuencia *un token a la vez*, el número de pasos de cómputo puede crecer con la longitud del programa. LPN, al condicionar en una sola pasada con un vector fijo, está atado a una cantidad constante de cómputo, sin importar cuán complejo sea el problema. Esto es lo que permite a NLI, en principio, resolver problemas *más difíciles* que los vistos en entrenamiento —algo imposible para un modelo de cómputo constante.

Por último, el paper motiva su enfoque también desde el ángulo de la *búsqueda combinatoria*: los métodos simbólicos clásicos necesitan funciones externas (aprendidas o heurísticas) que “guíen” la búsqueda entre las combinaciones posibles de un DSL, porque el espacio crece exponencialmente con la longitud del programa. NLI, en cambio, no separa “la guía” de “el lenguaje”: al ser diferenciable de punta a punta, la guía de búsqueda *es* el gradiente de la propia red, calculado directamente sobre el espacio de programas relajado. Esto es análogo a hacer búsqueda local en un espacio simbólico, pero con la enorme ventaja de tener una señal de gradiente en vez de tener que evaluar candidatos a ciegas.

¿Por qué importa esto?

Esta idea —reemplazar una búsqueda combinatoria discreta por una optimización continua diferenciable— es un patrón general en IA moderna, compartido por ejemplo con arquitecturas de búsqueda neuronal de arquitecturas (*Neural Architecture Search*) o con el uso de relajaciones continuas en optimización combinatoria clásica (como relajaciones LP de problemas enteros). Cada vez que veas un problema “imposible de enumerar” resuelto con gradientes, probablemente hay una relajación continua similar a la de NLI detrás.

Para pensar

LPN sacrifica composicionalidad a cambio de resistencia al sobreajuste (overfitting). ¿Por qué crees que comprimir todo el programa en un solo vector continuo protege contra el sobreajuste? Y al pasar a una secuencia discreta de tokens en NLI, ¿qué mecanismo (vas a verlo en la Sección 4.1) hace falta para no perder esa misma protección?

3 Planteamiento del problema (Problem Statement)

Antes de diseñar cualquier modelo, hay que dejar clarísimo *qué problema exactamente* se está resolviendo. Esta sección del paper es corta pero fundamental: define formalmente la tarea, llamada **inducción de programas** (*program induction*), en el marco de **programación por ejemplos** (*Programming by Example*, PBE).

Programming by Example, PBE

Un paradigma en el que la única información disponible sobre un programa desconocido es un conjunto de pares de *entrada-salida* que ese programa produce. Nunca se observa el código del programa en sí, solo su comportamiento. El objetivo es predecir la salida para entradas nuevas, *sin* necesariamente reconstruir el código fuente explícito.

Formalicemos, con la notación exacta del paper:

Desglosando la ecuación: el modelo $M(S, x_{n+1})$

Existe un programa desconocido p , tomado de un espacio de programas posibles $\mathcal{P}$. Se generan $n + 1$ entradas $\{x_i\}_{i=1}^{n+1} \sim P(X \mid p)$ (una distribución de entradas que depende del propio programa: por ejemplo, si p opera sobre listas de enteros, las entradas serán listas de enteros).

- Las primeras n parejas forman la **especificación** $S = \{(x_i, p(x_i))\}_{i=1}^n$: esto es literalmente “los ejemplos” que el modelo puede observar.
- La entrada x_{n+1} es la **consulta** (*query*): un caso nuevo, no visto, para el cual queremos predecir la salida.
- Un modelo M recibe ambos, S y x_{n+1} , y produce una predicción $\hat{y}_{n+1} = M(S, x_{n+1})$.
- El objetivo de entrenamiento es que esa predicción se acerque lo más posible a la verdadera salida $p(x_{n+1})$, que en entrenamiento sí conocemos (porque generamos los datos nosotros), pero que el modelo **nunca** tiene acceso directo a p en sí, ni en entrenamiento ni en prueba.

Un punto crucial, remarcado explícitamente en el paper: el modelo jamás ve una representación explícita de p (como podría ser su código fuente o su árbol sintáctico). Esto es deliberado y realista: en el mundo real, casi nunca tenemos acceso al “programa verdadero” detrás de un comportamiento que queremos imitar o predecir —solo vemos ejemplos de lo que hace.

Finalmente, para que esto sea un experimento de **generalización** y no solo de memorización, se distingue entre la distribución de entrenamiento P_{train} y la de prueba P_{test} , ambas sobre el espacio $\mathcal{P}$ de programas posibles. Si P_{test} es igual a P_{train} , hablamos de generalización **in-distribution** (ID). Si P_{test} contiene programas de un tipo distinto o más complejo que los vistos en entrenamiento, hablamos de generalización **out-of-distribution** (OOD) — y este es precisamente el tipo de generalización que más le interesa a este paper.

Intuición: un ejemplo numérico concreto

Supongamos que el programa oculto es $p(x) = \text{shift_left}(x, 2)$, es decir, “rotar la lista dos posiciones hacia la izquierda”. La especificación S podría ser:

$$S = \{ ([1, 2, 3, 4], [3, 4, 1, 2]), ([5, 6, 7, 8], [7, 8, 5, 6]) \}$$

Y la consulta podría ser $x_{n+1} = [9, 10, 11, 12]$. El modelo nunca ve el texto “`shift_left(x, 2)`”; solo ve estos pares de listas. Su tarea es predecir $\hat{y}_{n+1} = [11, 12, 9, 10]$ a partir de S y x_{n+1} , sin que nadie le haya dicho jamás qué operación se está aplicando.

¿Por qué importa esto?

PBE es exactamente el marco detrás de funcionalidades como el *Flash Fill* de Microsoft Excel: el usuario escribe un par de ejemplos de cómo transformar una columna de texto, y el programa infiere la regla general para aplicarla al resto de las filas, sin que el usuario escriba

código. Cualquier sistema que “aprenda una regla a partir de pocos ejemplos” —autocompletado inteligente, generación de reglas de validación de datos, algunos asistentes de programación— vive en este mismo marco formal.

Para pensar

Si el modelo nunca observa el programa verdadero p , ¿en qué sentido podemos decir que el modelo “induce un programa”? ¿Es posible que $M(S, x_{n+1})$ prediga correctamente $p(x_{n+1})$ sin haber aprendido nada parecido a un programa interno, sino simplemente una función estadística que “da la de casualidad”? Guarda esta pregunta: la Sección 5.2 del paper da evidencia experimental directamente relevante para responderla.

4 Inferencia secuencial discreta: el corazón de NLI

Esta es la sección técnica central del paper. La dividimos exactamente como el original: objetivo de entrenamiento, inductor (encoder), intérprete (decoder) y búsqueda en tiempo de prueba. Tómate tu tiempo aquí: si entiendes bien esta sección, el resto del paper (experimentos, ablaciones) se lee casi solo, porque son evidencia de que estas piezas funcionan.

NLI es, en su forma más básica, una arquitectura **encoder-decoder**:

- El **encoder**, al que llamaremos *inductor* q_ϕ , mira la especificación S y produce una representación latente del programa.
- El **decoder**, al que llamaremos *intérprete* p_θ , toma esa representación y la ejecuta sobre una entrada dada para predecir una salida.

4.1 Objetivo de entrenamiento (*Training Objective*)

¿Cómo se entrena esto para que el inductor aprenda a producir programas *útiles* y no cualquier secuencia de tokens? La respuesta del paper tiene dos ingredientes: una estrategia de muestreo llamada **leave-one-out**, y una función de pérdida con dos términos.

Intuición: por qué leave-one-out

Si el inductor pudiera ver el mismo par (x_i, y_i) que después se le pide predecir, el modelo podría hacer trampa: le bastaría con “memorizar y repetir” ese par, sin haber entendido la regla general detrás. Para evitar esto, en cada especificación de m pares, se le esconde uno al inductor (el par i), se le pide que infiera el programa con los $m - 1$ pares restantes, y luego se evalúa si, con ese programa inferido, puede predecir correctamente la salida del par que se guardó. Es exactamente la misma lógica que usar un conjunto de validación: forzar al modelo a generalizar, no a memorizar.

Desglosando la ecuación: Ecuación 1: el objetivo completo

$$\mathcal{L}(\phi, \theta; S) = \frac{1}{B} \sum_{b=1}^B \left(\frac{1}{m} \sum_{i=1}^m \mathcal{L}_{\text{recon}}(\phi, \theta; x_{b,i}, y_{b,i}, S_{b,i}) + \lambda_{\text{reg}} \cdot \mathcal{L}_{\text{reg}}(\phi; S_b) \right)$$

Leyendo de afuera hacia adentro:

- $\frac{1}{B} \sum_{b=1}^B (\cdot)$: promediamos sobre las B especificaciones del *mini-batch* (un lote de entre-

namiento contiene B tareas distintas a la vez, no solo una).

- $\frac{1}{m} \sum_{i=1}^m (\cdot)$: dentro de cada especificación S_b (que tiene m pares entrada-salida), aplicamos la estrategia leave-one-out a *cada uno* de los m pares, uno a la vez, y promediamos. $S_{b,i} = S_b \setminus \{(x_{b,i}, y_{b,i})\}$ es la especificación b sin el par i (la notación “ $\setminus$ ” significa “conjunto sin este elemento”).
- $\mathcal{L}_{\text{recon}}$: la pérdida de *reconstrucción*, que mide qué tan bien el modelo predice el par escondido.
- $\lambda_{\text{reg}} \cdot \mathcal{L}_{\text{reg}}$: un término de *regularización*, ponderado por un hiperparámetro λ_{reg} , que empuja al inductor a reutilizar tokens en vez de inventar uno nuevo para cada tarea.

Desglosando la ecuación: Ecuación 2: pérdida de reconstrucción

$$\mathcal{L}_{\text{recon}}(\phi, \theta; x, y, S) = -\log p_{\theta}(y \mid x, q_{\phi}(S))$$

Esto es simplemente la **log-verosimilitud negativa** de la salida correcta y , dado que el intérprete p_{θ} ejecuta la entrada x condicionado en el programa latente que el inductor infirió a partir de la especificación, $q_{\phi}(S)$. Es la misma idea que la entropía cruzada usada para entrenar casi cualquier clasificador: cuanto más probabilidad le asigna el modelo a la respuesta correcta, más chica es esta pérdida (el logaritmo de un número cercano a 1 es cercano a 0; el logaritmo de un número cercano a 0 tiende a $-\infty$, así que el signo negativo hace que minimizar la pérdida equivalga a maximizar esa probabilidad).

Desglosando la ecuación: Ecuación 3: pérdida de reutilización de tokens

$$\mathcal{L}_{\text{reg}}(\phi; S) = \sum_{k=1}^K \sum_{j=1}^N \left[1 - \exp \left(\sum_{b=1}^B \sum_{i=1}^m \log \left(1 - q_{\phi}^{(k,j)}(S_{b,i}) \right) \right) \right]$$

Esta fórmula asusta más de lo que es. Vamos por partes:

- $q_{\phi}^{(k,j)}(S_{b,i})$ es la probabilidad que el inductor le asigna al token k (de un vocabulario de tamaño K) en la posición j del programa, dado el ejemplo $S_{b,i}$.
- El término $1 - q_{\phi}^{(k,j)}(S_{b,i})$ es la probabilidad de que el token k *no* aparezca en esa posición concreta.
- Al sumar los logaritmos de esas probabilidades sobre todas las posiciones j y todas las especificaciones del batch, y luego aplicar $\exp(\cdot)$, se obtiene (bajo un supuesto de independencia simplificador) la probabilidad de que el token k **nunca** aparezca en absoluto en todo el lote.
- Entonces $1 - \exp(\cdot)$ es la probabilidad de que el token k aparezca **al menos una vez** en el lote.
- Sumando esto sobre los K tokens del vocabulario, obtenemos una aproximación diferenciable al *número esperado de tokens distintos usados* en el lote.

Minimizar $\mathcal{L}_{\text{reg}}$ empuja a que ese número esperado sea lo más chico posible: el modelo es castigado por usar muchos tokens distintos, y premiado por reutilizar unos pocos una y otra vez.

Intuición: por qué queremos reutilización

Imaginate un lenguaje de programación con una palabra nueva para cada tarea distinta que resuelve: técnicamente “funciona”, pero no aprendiste nada generalizable, solo memorizaste un diccionario gigante de casos particulares (exactamente la enfermedad que le achacábamos a LPN). Si en cambio el modelo es castigado por inventar demasiadas palabras nuevas, se ve forzado a encontrar un vocabulario chico de primitivas que, combinadas de distintas formas,

alcancen para resolver muchas tareas. Eso es, literalmente, lo que hace que un lenguaje sea composicional.

¿Por qué importa esto?

Este tipo de regularización —penalizar la cantidad de “símbolos distintos” usados para forzar representaciones compactas y reutilizables— aparece bajo otros nombres en distintas áreas: *sparse coding* en visión, *minimum description length* en teoría de la información, y el objetivo de compresión de vocabulario en tokenizadores de lenguaje natural. La intuición de fondo —“menos símbolos, mejor generalización”— es una de las ideas más transversales en aprendizaje de representaciones.

Para pensar

El propio paper reconoce (en su apéndice) que esta pérdida se minimiza trivialmente colapsando *todo* a un único token —el vocabulario más chico posible es de tamaño 1—. ¿Por qué crees que esto no ocurre en la práctica? Pista: piensa en qué le pasaría a $\mathcal{L}_{\text{recon}}$ si el modelo intentara resolver *todas* las tareas con un solo token.

4.2 El inductor: aprendiendo una representación discreta del programa

El inductor q_ϕ es una función que mapea una especificación S (un conjunto de pares entrada-salida) a una secuencia latente $z = (z_1, \dots, z_T)$ de longitud fija T , donde cada z_t es una **distribución de probabilidad** sobre un vocabulario (“codebook”) de K posibles tokens, es decir $z_t \in \Delta^{K-1}$ (el simplex de probabilidad de dimensión $K - 1$: un vector de K números no negativos que suman 1).

codebook y token de skip

El **codebook** es simplemente el vocabulario aprendido: un conjunto de K “instrucciones” abstractas que el modelo puede usar para construir programas. Un token especial dentro de ese vocabulario es el **token de skip**: le indica al intérprete “no hagas nada en este paso”, y es lo que permite representar programas más cortos que la longitud máxima fija T , simplemente rellenando el resto con skips.

El inductor construye esta secuencia en dos pasos.

Paso 1 — embeber cada ejemplo y agregarlos. Un transformer h_ϕ se aplica *independientemente* a cada par (x_i, y_i) de la especificación, produciendo T embeddings de token por cada par. Como la especificación tiene varios pares y el orden en que llegan no debería importar (dos ejemplos iguales en distinto orden describen el mismo programa), se promedian (*mean-pooling*) las representaciones de todos los pares, posición por posición:

$$\bar{e}_t = \frac{1}{|S|} \sum_{i=1}^{|S|} e_t^{(i)}$$

Intuición: por qué promediar y no concatenar

Si en cambio concatenáramos los ejemplos en un orden fijo, el modelo podría aprender (sin querer) que “el primer ejemplo importa más”, o directamente fallar si en test le damos los ejemplos en otro orden o en otra cantidad. Promediar garantiza que el resultado sea **invariante**

a permutaciones: da lo mismo el orden en que llegan los ejemplos, y de paso el modelo puede generalizar a especificaciones con más o menos ejemplos de los vistos en entrenamiento. Esta misma idea de “agregar por promedio para lograr invariancia al orden” es el principio de diseño detrás de los *Neural Processes*.

Paso 2 — de embedding continuo a distribución discreta. Cada embedding agregado $\bar{e}_t$ se pasa por una red feed-forward f_ϕ compartida, que produce K *logits* (puntuaciones sin normalizar, una por cada token del vocabulario). El problema es que, para poder entrenar con descenso de gradiente, necesitamos que el paso de “elegir un token discreto” sea diferenciable —y elegir el máximo (**argmax**) de un vector de logits **no** es diferenciable (es una función escalonada, con gradiente cero casi en todas partes). Acá es donde entra la relajación de **Gumbel-Softmax**.

Intuición: Gumbel-Softmax: el dado que se enfría

Imaginate que en vez de tirar un dado normal (discreto, resultado exacto) tenés un dado “de gelatina”: al principio del entrenamiento está tibio y blando, y cuando lo tirás, en vez de caer en un solo número, se reparte *parcialmente* entre varias caras a la vez (por ejemplo, 40 % cara 3, 35 % cara 5, el resto repartido). A medida que el entrenamiento avanza, vamos “enfriando” el dado (bajando un parámetro llamado *temperatura*, τ), y se va poniendo cada vez más rígido, hasta que casi siempre cae, prácticamente, en una sola cara. Gumbel-Softmax es exactamente ese dado de gelatina: una forma de *simular* un muestreo discreto (de una distribución categórica) que sigue siendo diferenciable, porque en cada paso intermedio produce una mezcla suave de opciones en vez de una elección dura.

Desglosando la ecuación: la fórmula de Gumbel-Softmax

$$l_t = f_\phi(\bar{e}_t) \in \mathbb{R}^K, \quad z_t = \text{Softmax}\left(\frac{l_t + g_t}{\tau_e}\right) \in \Delta^{K-1}$$

donde $g_t \sim \text{Gumbel}(0, 1)$ es ruido aleatorio muestreado de una distribución Gumbel (una distribución diseñada específicamente para que, sumada a los logits y pasada por un **argmax**, reproduzca exactamente un muestreo de la categórica original), y $\tau_e > 0$ es la temperatura del inductor.

- Cuando τ_e es **alto**, dividir por τ_e “aplasta” las diferencias entre logits, y el Softmax resultante es una distribución *suave*, casi uniforme: el dado de gelatina está blando, hay mucha exploración.
- Cuando $\tau_e \rightarrow 0$, dividir por τ_e *amplifica* enormemente las diferencias entre logits, y el Softmax se acerca a un **argmax** verdadero (casi toda la probabilidad se concentra en un solo token): el dado se congela y el resultado es, en la práctica, discreto.

Durante el entrenamiento, τ_e se **recuece** (*annealing*): empieza alto y se reduce gradualmente, para que el modelo primero explore libremente y termine comprometiéndose a activaciones discretas y “puntudas” del codebook.

¿Por qué importa esto?

Gumbel-Softmax es una herramienta de propósito general para cualquier situación en la que queremos que una red neuronal *elija* entre opciones discretas mientras se sigue entrenando con backpropagation: selección de expertos en modelos *Mixture-of-Experts*, selección de arquitectura en *Neural Architecture Search*, generación de texto diferenciable, y —como acá— generación

de tokens de un lenguaje de programa. Si en algún momento ves un modelo que necesita “decidir algo discreto” pero se entrena de punta a punta con gradientes, es muy probable que use Gumbel-Softmax o una alternativa cercana (como VQ-VAE con *straight-through estimator*, que discutiremos en la Sección 5.3).

Para pensar

Si enfriamos la temperatura demasiado rápido, el modelo se ve forzado a comprometerse con tokens discretos antes de haber explorado bien el espacio de posibles vocabularios. El propio paper reporta (en un apéndice) que un recocido demasiado agresivo degrada fuertemente el desempeño. ¿Por qué crees que ocurre esto? ¿Qué tiene esto en común con el problema de bajar la temperatura demasiado rápido en algoritmos de *simulated annealing* clásicos?

4.3 El intérprete: ejecución neuronal recurrente de programas

El intérprete p_θ es la pieza que le da a NLI su nombre: es literalmente un “intérprete” de un lenguaje, en el mismo sentido que un intérprete de Python o de Lisp: recibe una secuencia de instrucciones (los tokens $z_1, \dots, z_T$ producidos por el inductor) y las ejecuta **una a la vez**, manteniendo un estado interno que va cambiando paso a paso.

Intuición: por qué recurrencia y no una sola pasada

Un decoder estándar (como el de LPN) suele condicionar toda la predicción en una única pasada hacia adelante, viendo el programa completo de una vez. El problema es que ese tipo de decoder tiende a sobreajustarse a las longitudes y estructuras de programa vistas en entrenamiento: si siempre vio programas de exactamente 4 pasos, le cuesta mucho generalizar a programas de 8 pasos. El intérprete de NLI, en cambio, usa una red de ejecución *compartida* d que se aplica repetidamente, un token a la vez, durante T pasos. Esto fuerza a la red a aprender un mecanismo de ejecución *reutilizable* —el mismo procedimiento sirve para el token 1, el token 2, ..., el token 50— en vez de aprender un mapeo monolítico específico para una longitud de programa fija.

Veamos el Algoritmo 1 del paper línea por línea. Los parámetros θ se descomponen en tres piezas: un *embedder de código* E_v (que convierte tokens del codebook en vectores), un *embedder de entrada-salida* E_{io} (que convierte valores del dominio, como listas de enteros, en vectores), y la *red de ejecución* d propiamente dicha.

Desglosando la ecuación: Algoritmo 1: ejecución del intérprete

```

1: function  $p_\theta(y \mid x, z, \tau, h)$ 
2:    $s_0 \leftarrow \text{Embed}(x, E_{io})$                                 ▷ estado inicial = embedding de la entrada
3:    $\pi_0 \leftarrow \text{OneHot}(x)$                                     ▷ distribución de salida inicial = la propia entrada
4:   for  $t = 1 \rightarrow T$  do
5:      $n_t \leftarrow d(s_{t-1}, E_v^\top z_t)$                         ▷ la red predice logits de salida
6:      $\tilde{\pi}_t \leftarrow \text{Softmax}((n_t + h_t)/\tau)$                 ▷ Gumbel-Softmax
7:      $\pi_t \leftarrow z_{t,\text{skip}} \cdot \pi_{t-1} + (1 - z_{t,\text{skip}}) \cdot \tilde{\pi}_t$     ▷ skip-gating
8:      $s_t \leftarrow E_{io}^\top \pi_t$                                 ▷ re-embeber para el próximo paso
9:   end for
10:  return  $\pi_T$                                                 ▷ distribución de salida final
11: end function

```

Línea por línea, en criollo:

- **Líneas 2-3.** El estado inicial s_0 es simplemente la entrada x , embebida en el espacio vectorial del intérprete. La distribución de salida inicial π_0 es un *one-hot* (todo el peso en un solo valor) que apunta exactamente a x : antes de ejecutar ningún token, “la mejor predicción posible” es no cambiar nada.
- **Línea 5.** En cada paso t , la red de ejecución d toma el estado anterior s_{t-1} y el token actual (embebido vía E_v) y produce logits n_t : una predicción de cómo debería verse la nueva salida tras aplicar esta instrucción.
- **Línea 6.** Igual que en el inductor, esos logits se combinan con ruido Gumbel h_t y se pasan por un Softmax con temperatura τ (la temperatura del intérprete, llamada τ_d en el resto del paper), para obtener una distribución candidata $\tilde{\pi}_t$ sobre posibles valores de salida.
- **Línea 7 — el corazón del mecanismo de skip.** Acá es donde entra el token de skip que vimos en la subsección anterior. $z_{t,\text{skip}}$ es la probabilidad (entre 0 y 1) de que el token actual sea “no-op”. La nueva distribución de salida π_t es una **interpolación lineal** entre la distribución anterior π_{t-1} (si el token es skip, no cambiamos nada) y la candidata $\tilde{\pi}_t$ (si el token es una instrucción real, aplicamos el cambio). Esto permite que el modelo, efectivamente, “ignore” pasos innecesarios sin tener que truncar la secuencia de forma discreta.
- **Línea 8.** El nuevo estado s_t se obtiene proyectando de vuelta la distribución de salida (ya actualizada) al espacio de estados, para que el próximo paso pueda seguir operando sobre ella.
- **Línea 10.** Después de los T pasos, la distribución final π_T es la predicción del modelo.

Intuición: analogía con un procesador de verdad

Este algoritmo es, en espíritu, muy parecido a cómo un procesador ejecuta instrucciones de máquina: hay un *estado* (los registros/la memoria) que se actualiza en cada ciclo de reloj, según la instrucción actual y el estado anterior. La diferencia es que acá tanto las “instrucciones” como el “estado” son vectores continuos aprendidos, y toda la ejecución es diferenciable —lo que en un procesador real sería impensable, pero es justamente lo que permite luego *buscar* programas con gradiente descendente.

¿Por qué importa esto?

La idea de un “ejecutor neuronal” que interpreta una secuencia de código, token a token, mientras mantiene un estado recurrente, conecta directamente con los *Neural Programmer-Interpreters* (Reed & De Freitas, 2015) y con los *world models* recurrentes usados en aprendizaje por refuerzo (Ha & Schmidhuber, 2018), donde una red aprende a simular la dinámica de un entorno paso a paso. Si te interesa robótica o control, este patrón de “red recurrente que actualiza un estado condicionada en una acción/instrucción” es exactamente el mismo que verás en modelos de dinámica de un entorno (*world models*).

Para pensar

Fijate que la línea 8 usa $E_{io}^\top \pi_t$, es decir, re-proyecta la distribución *completa* π_t (no un token discreto muestreado) de vuelta al espacio de estados. ¿Por qué crees que es importante mantener esto como una distribución suave en vez de forzar una elección discreta en cada paso intermedio, incluso aunque al final del proceso el objetivo sea obtener una predicción concreta?

4.4 Búsqueda de programas neuronales en tiempo de prueba

Esta es, para muchos, la contribución más distintiva del paper: el inductor da una *primera adivinanza* razonable, pero no necesariamente óptima —especialmente para programas fuera de la distribución de entrenamiento. NLI aprovecha que **todo el sistema es diferenciable** para *refinar* esa adivinanza con descenso de gradiente, en tiempo de prueba, sin necesidad de reentrenar nada.

Intuición: la idea central en una frase

En vez de buscar el mejor programa *enumerando* combinaciones discretas (como haría un sintetizador simbólico clásico, que tiene que probar candidato por candidato), NLI busca en una versión **continua y relajada** del espacio de tokens —los mismos vectores de embeddings e que alimentan al Gumbel-Softmax— y usa el gradiente de la verosimilitud respecto a esos vectores para moverse cuesta arriba, hacia programas que expliquen mejor los ejemplos observados.

Desglosando la ecuación: el objetivo de búsqueda

$$e^* = \operatorname{argmax}_e \mathbb{E}_{\substack{g \sim \text{Gumbel}(0,1)^T \\ h \sim \text{Gumbel}(0,1)^T}} \left[\sum_{(x_j, y_j) \in S} \log p_\theta(y_j \mid x_j, z(e, \tau_e, g), \tau_d, h) \right]$$

- Buscamos los embeddings continuos e (no los tokens discretos directamente) que **maximicen** la verosimilitud esperada de toda la especificación S .
- La expectativa $\mathbb{E}[\cdot]$ está tomada sobre el ruido Gumbel tanto del inductor (g) como del intérprete (h): como el muestreo es estocástico (recordá el “dado de gelatina”), promediamos sobre distintas realizaciones del ruido para que la búsqueda no se ajuste a una única muestra de ruido particular.
- $z(e, \tau_e, g)$ es exactamente la función de Gumbel-Softmax que vimos en la Sección 4.2: convierte el embedding continuo e en la secuencia de distribuciones categóricas $z_{1:T}$.
- En criollo: “movete en la dirección de e que hace que el intérprete, ejecutando el programa resultante, prediga mejor los ejemplos que tengo”.

Igual que en el entrenamiento, se aplica **recocido de temperatura** tanto a τ_e como a τ_d durante la optimización: empezar con temperatura alta para explorar libremente el espacio (suavizando el objetivo, evitando quedar atrapados en mínimos locales muy afilados prematuramente), y bajarla gradualmente para terminar comprometidos a una representación discreta y nítida. Al final de la búsqueda, curiosamente, **no** se fuerza un muestreo discreto puro: se usa la representación final *suave* (soft) para la ejecución definitiva, evitando así introducir una discrepancia entre lo que se optimizó y lo que finalmente se ejecuta.

Intuición: múltiples arranques, como una carrera en paralelo

En vez de arrancar la búsqueda desde un único punto (el embedding producido por el inductor), NLI muestrea s puntos de partida distintos a partir de una Gaussiana centrada en ese embedding ($\mu = e_t$, $\sigma = \sigma_s$), y corre L pasos de ascenso de gradiente **en paralelo** desde cada uno de ellos. Es como mandar varios exploradores en direcciones ligeramente distintas alrededor del punto de partida, para no quedar atrapados en un mínimo local si el punto de partida original no era muy bueno. Como estas búsquedas son independientes entre sí, se pueden repartir entre distintos dispositivos de hardware (GPUs/TPUs) y correr todas a la vez.

Un detalle importante que el paper señala explícitamente: en la mayoría de los métodos de sínte-

sis de programas, existe el riesgo de encontrar *múltiples* programas que explican igual de bien la especificación observada, pero que no generalizan de la misma forma a datos nuevos —el clásico problema de **sobreajuste a la especificación**. En espacios de búsqueda puramente continuos (como el de LPN), esto suele requerir trucos como *early stopping* para no converger a soluciones demasiado específicas. NLI argumenta que este riesgo es menor en su caso, porque el espacio de búsqueda —aunque relajado— sigue estando limitado a *recomponer un conjunto discreto de embeddings aprendidos*, lo cual es intrínsecamente menos flexible (y por lo tanto menos propenso a sobreajustar) que un espacio continuo sin restricciones.

¿Por qué importa esto?

Esta técnica de “optimizar por gradiente una relajación continua de un problema discreto, en tiempo de inferencia, con múltiples arranques aleatorios” es prácticamente un patrón de diseño en sí mismo. Aparece, por ejemplo, en la optimización de *prompts* continuos (*prompt tuning*), en ataques adversarios contra redes neuronales (donde se busca una perturbación de entrada que engañe al modelo), y en diseño de moléculas mediante optimización en espacios latentes continuos de modelos generativos. Si entendiste esta sección, ya tenés la base conceptual para entender esas otras aplicaciones.

Para pensar

El paper describe esta búsqueda como “análoga a la búsqueda local en espacios simbólicos, pero con la ventaja de tener señales de gradiente”. ¿En qué se parece esto a algoritmos de búsqueda local clásicos, como *hill climbing* o *simulated annealing*? ¿Y en qué se diferencia fundamentalmente, dado que acá sí tenemos una dirección de mejora (el gradiente) en vez de tener que evaluar vecinos a ciegas?

5 Experimentos

Hasta ahora vimos *cómo está construido* NLI. Esta sección responde a la pregunta obligada: ¿realmente funciona, y por qué? Los autores diseñan experimentos muy específicamente pensados para aislar la capacidad de **generalización composicional** —no solo “qué tan bien predice en promedio”, sino “qué tan bien recombina piezas aprendidas para resolver tareas estructuralmente nuevas”.

5.1 Generalización composicional: los benchmarks Shift-L, Shift-P y Comp-I

Los autores construyen un banco de pruebas propio, con secuencias de longitud fija (20), diseñado específicamente para exponer fallas de generalización composicional en el marco PBE. Tiene tres particiones:

- **Shift-L** (generalización de longitud). Entrena con `shift_left(n)` para $n \in \{1, \dots, 5\}$, evalúa con $n \in \{6, \dots, 10\}$. Mide si el modelo extrapola la *magnitud* de un parámetro más allá de lo visto.
- **Shift-P** (extracción de sub-funciones). Es el experimento inverso: entrena con desplazamientos *grandes* ($n \in \{7, 8, 9\}$) y evalúa con desplazamientos *chicos* ($n \in \{1, 2, 3\}$). La pregunta aquí es sutil: ¿aprendió el modelo el concepto abstracto de “desplazar por n ”, del cual un desplazamiento chico es un caso particular, o memorizó programas completos específicos para cada n grande visto?

- **Comp-I** (composición de primitivas). El modelo ve, en entrenamiento, más de 20 primitivas *aisladas* (por ejemplo, `reverse`, `shift_left_3`), nunca combinadas. En evaluación, debe ejecutar *composiciones* de dos o más primitivas, del estilo $f(g(x))$, nunca vistas juntas durante el entrenamiento.

Intuición: por qué estos tres, y no uno solo

Cada benchmark ataca un tipo distinto de composicionalidad: Shift-L prueba extrapolar *magnitud* (más de lo mismo, pero más grande), Shift-P prueba si existe un concepto abstracto reutilizable detrás de casos memorizados, y Comp-I prueba la composición *estructural* de piezas nunca vistas juntas. Un modelo podría, en principio, pasar uno de estos tres sin realmente tener generalización composicional (por ejemplo, memorizando patrones de bajo nivel); que NLI pase los tres simultáneamente es una evidencia mucho más fuerte.

Los resultados (Tabla 1 del paper) son contundentes: **todos** los métodos —incluyendo In-Context Learning, Test-Time Training (TTT), y Latent Program Networks (LPN y su variante discreta D-LPN)— alcanzan casi 100 % de precisión *in-distribution* (ID), pero colapsan a prácticamente 0 % *out-of-distribution* (OOD) en las tareas de Shift. Incluso NLI *sin* búsqueda por gradiente (solo con la inferencia directa del inductor) falla en OOD de forma similar. La diferencia dramática aparece con **NLI + Gradient Search**: 99 % en Shift-L OOD, 100 % en Shift-P OOD, y 91 % en Comp-I OOD.

¿Por qué importa esto?

Este resultado es la evidencia empírica central del paper: no basta con tener una representación discreta y composicional *en principio* (el inductor solo ya la tiene, y aun así falla en OOD); hace falta, además, el paso de **búsqueda activa en tiempo de prueba** para que esa estructura se aproveche en problemas realmente nuevos. Es un recordatorio importante: tener la *capacidad representacional* correcta no garantiza que el modelo la use bien por defecto; a veces hace falta un mecanismo de inferencia adicional (como aquí, optimización en tiempo de prueba) para explotarla.

Un detalle revelador (Figura 2 del paper, benchmark Shift-L): el modelo aprende, por sí solo, un token (llamado 231 en el paper) que representa “un desplazamiento simple hacia la izquierda”, y un segundo token (476) para “dos desplazamientos”. Combinándolos, representa desplazamientos de 4 y 5 posiciones vistos en entrenamiento; y al generalizar OOD a un desplazamiento de 8 (nunca visto), el modelo *compone* esos mismos dos tokens de una forma nueva (231 231 231 231 476 476) en vez de necesitar un token nuevo. Esto es composicionalidad genuina, observable directamente en la representación aprendida.

Para pensar

Los baselines TTT y LPN también tienen, en principio, mecanismos de adaptación en tiempo de prueba (ajuste de parámetros o búsqueda en un espacio latente continuo, respectivamente), y aun así fallan en OOD casi por completo. ¿Qué hace que la búsqueda de NLI sea distinta y efectiva donde estas otras formas de adaptación no lo son? Pista: pensá en la diferencia entre *qué* se está buscando en cada caso (pesos del modelo vs. un vector continuo sin estructura vs. una secuencia discreta composicional).

5.2 Ablaciones: ¿qué componente es realmente responsable del éxito?

Un modelo puede tener muchas piezas móviles; una **ablación** consiste en quitar una pieza a la vez y medir cuánto se degrada el desempeño, para saber cuáles son realmente necesarias y cuáles son solo “adornos”.

Estudio de ablación

Metodología experimental en la que se comparan versiones del modelo completo contra versiones a las que se les remueve, de a una, cada componente candidato, manteniendo todo lo demás igual. La caída (o no) de desempeño al remover un componente es evidencia de cuán *causalmente necesario* es ese componente para el resultado observado.

El modelo base de NLI alcanza 97 % de precisión OOD. Al remover componentes uno por uno, los resultados (Figura 3 del paper) muestran una jerarquía clara de importancia:

- Remover la **ejecución recurrente** o la **discreción** (tanto del inductor como del intérprete) hace colapsar la precisión OOD a apenas 1–5 %. Estos son, sin duda, los ingredientes indispensables.
- Remover el **token de skip dedicado** baja el desempeño a 24 %: el modelo puede, en teoría, aprender su propio mecanismo de “no-op” sin un token especial, pero tenerlo explícito acelera y estabiliza mucho el entrenamiento.
- Remover la **pérdida de regularización** ($\mathcal{L}_{\text{reg}}$, la de la Sección 4.1) provoca una caída más moderada, y su importancia depende del *tipo* de composicionalidad que se esté probando: es crucial para Shift-P (donde hace falta identificar primitivas reutilizables detrás de programas largos), pero casi irrelevante para la generalización de longitud pura.

Intuición: cómo leer un gráfico de ablación

Cuanto más cae el desempeño al remover un componente, más “responsable” es ese componente del comportamiento observado. Si remover algo *no* cambia mucho el resultado, ese componente probablemente no era la causa real del éxito —aunque estuviera presente en el modelo que funcionaba bien. Es la diferencia entre correlación (“el modelo exitoso tenía este componente”) y causalidad (“este componente es lo que hace exitoso al modelo”).

5.3 El rol del Gumbel-Softmax

Profundizando en la ablación anterior, remover el muestreo Gumbel a nivel del **intérprete** es catastrófico (cae a 5 %), mientras que removerlo a nivel del **inductor** es menos severo pero igual importante (de 97 % a 76 %). Curiosamente, el inductor sigue produciendo una distribución discreta incluso sin Gumbel-Softmax —los autores conjeturan que la red puede aprender a producir distribuciones “puntudas” por sí sola—, pero la relajación de Gumbel-Softmax refuerza ese sesgo inductivo de forma mucho más contundente.

Un punto que vale la pena resaltar por honestidad científica: los propios autores aclaran que su método **no depende fundamentalmente** de Gumbel-Softmax en particular; cualquier relajación diferenciable de un muestreo discreto podría, en principio, sustituirla —por ejemplo, un **VQ-VAE** con *straight-through estimator*. Eligieron Gumbel-Softmax sobre todo por estabilidad de entrenamiento: el straight-through estimator de VQ-VAE es conocido por sufrir gradientes sesgados y “colapso del codebook” (donde muchas entradas del vocabulario quedan sin usarse nunca). El costo

de Gumbel-Softmax es que requiere un recocido de temperatura cuidadoso; recocer demasiado rápido degrada severamente el desempeño.

¿Por qué importa esto?

Esta comparación entre Gumbel-Softmax y VQ-VAE es una de las decisiones de diseño más comunes al construir cualquier sistema con variables discretas diferenciables (tokenizadores de imágenes, generadores de audio discreto, modelos de lenguaje con memoria discreta). No hay una respuesta única “mejor”: es un trade-off entre estabilidad de entrenamiento (a favor de Gumbel-Softmax) y sesgo de gradiente/colapso de codebook (el problema típico de VQ-VAE).

5.4 Escalando la búsqueda en tiempo de prueba

Si la búsqueda por gradiente es la clave del éxito, una pregunta natural es: ¿más cómputo en tiempo de prueba se traduce en mejor desempeño? Los autores exploran dos ejes independientes en el benchmark Comp-I: el número de **pasos de gradiente** por búsqueda, y el número de **arranques paralelos** (*num starts*, ver Sección 4.4).

El resultado (Figura 4 del paper) muestra una correlación positiva y consistente entre cómputo de test-time y precisión, en ambos ejes: más pasos de gradiente ayudan, y más arranques paralelos también ayudan, independientemente.

¿Por qué importa esto?

Esto conecta a NLI con una tendencia mucho más amplia en la IA reciente: el **escalado de cómputo en tiempo de inferencia** (*test-time compute*), la misma idea de fondo detrás de técnicas como *chain-of-thought* extendido o búsqueda en árbol en modelos de lenguaje grandes que “piensan más” antes de responder. La lección general es: si un modelo tiene un mecanismo genuino de búsqueda/refinado en tiempo de prueba (y no solo una pasada directa), invertir más cómputo ahí suele pagar dividendos de forma predecible.

Para pensar

Si más pasos de gradiente y más arranques paralelos ayudan de forma independiente, ¿qué estrategia de asignación de presupuesto computacional elegirías en la práctica: pocos arranques con muchos pasos cada uno, o muchos arranques con pocos pasos cada uno? ¿Qué factores (por ejemplo, disponibilidad de hardware paralelo) cambiarían tu respuesta?

5.5 DeepCoder: comparación con métodos neuro-simbólicos

Para probar que NLI escala más allá de un benchmark hecho a medida, los autores lo evalúan en **DeepCoder**, un benchmark estándar de síntesis de programas que manipulan listas de enteros mediante un DSL con operaciones de primer orden (**Sort**, **Sum**, **Take**, ...) y funcionales de orden superior (**Map**, **Filter**, **Scan1**, ...) que a su vez toman pequeñas funciones lambda como argumento (por ejemplo `+1`, `*2`).

El benchmark define cinco tipos de splits de generalización composicional (extrapolación de longitud de programa, composición de conceptos distintos, cambio de orden de composición, composición de una operación nueva, y extensión de funcionalidad de un operador conocido a argumentos nuevos).

La comparación es especialmente interesante porque incluye métodos **neuro-simbólicos** (ExeDec,

Transformer Program Synthesis, Latent Programmer) que tienen una ventaja injusta: se entrenan con acceso al *programa verdadero* (ground truth) como señal de supervisión adicional. NLI y LPN, en cambio, se entrenan *solo* con pares entrada-salida, sin ver jamás el programa correcto (fieles al planteamiento del problema de la Sección 3).

A pesar de esa desventaja de información, NLI (y LPN) superan ampliamente a los enfoques neuronales puramente basados en *Latent Programmer* y a los transformers estándar de síntesis de programas, y alcanzan un desempeño **competitivo** con ExeDec, el método neuro-simbólico de referencia, pese a no tener acceso al programa verdadero. Comparando NLI contra LPN directamente: NLI generaliza mejor a programas más largos y a composiciones de conceptos nuevas (su fortaleza natural, dada su ejecución recurrente de longitud variable), mientras que LPN es mejor cambiando el orden de composición y extendiendo la funcionalidad de un operador ya conocido —sugiriendo que ambas arquitecturas capturan sesgos inductivos complementarios, no estrictamente uno superior al otro en todo sentido.

¿Por qué importa esto?

Este resultado importa mucho para la pregunta de si vale la pena, en la práctica, invertir en anotar programas verdaderos (costoso y a veces imposible en dominios reales) versus invertir en arquitecturas que induzcan estructura solo desde ejemplos entrada-salida (como NLI). Que NLI se acerque a métodos supervisados con programas reales, sin usar esa supervisión, es evidencia de que gran parte de la estructura composicional de un dominio se puede *descubrir* en vez de tener que *especificarse* a mano.

6 Trabajo relacionado

Esta sección del paper sitúa a NLI dentro de un mapa más amplio de ideas. No hace falta memorizar cada cita, pero sí vale la pena entender las “familias” de trabajo, porque te van a servir como vocabulario para leer otros papers del área.

- **Síntesis simbólica de programas.** Trabajos clásicos (Summers, 1977; Gulwani, 2011) diseñan DSLs a mano y buscan exhaustivamente (o con heurísticas) dentro de ellos. Precisos e interpretables, pero no escalan a dominios nuevos sin rediseño manual. Los híbridos neuro-simbólicos como **DeepCoder** (Balog et al., 2017) combinan una red que *predice* qué componentes del DSL son probables, acelerando la búsqueda simbólica —pero siguen atados a un DSL fijo, diseñado por humanos.
- **Inducción neuronal y meta-aprendizaje.** Los *Neural Programmer-Interpreters* (Reed & De Freitas, 2015) ejecutan programas de forma implícita con modelos recurrentes; el *meta-induction* (Devlin et al., 2017) ataca el problema en un régimen de pocos ejemplos (*few-shot*). **ExeDec** (Shi et al., 2023) —el baseline neuro-simbólico usado en la Sección 5.5— agrega *descomposición de ejecución* como sesgo inductivo, pero sigue dependiendo de descomposiciones de referencia (ground truth) costosas de obtener. El **meta-aprendizaje** en general (Finn et al., 2017, con MAML como referencia clásica) entrena redes para adaptarse rápido a través de una distribución de tareas, un marco cercano al que usa NLI para entrenar su búsqueda de test-time.
- **Representaciones latentes de programas.** **CompILE** (Kipf et al., 2019) segmenta demostraciones en códigos latentes reutilizables usando Gumbel-Softmax, pero para *imitation learning*, no PBE. El **Latent Programmer** (Hong et al., 2021) usa un autoencoder estilo VQ-VAE con *beam search* en el espacio latente. Y, por supuesto, **LPN** (Macfarlane & Bonnet, 2025), el predecesor

directo de NLI discutido en la Sección 2, que introduce el paradigma de buscar en un espacio latente *continuo* en tiempo de prueba.

- **Composicionalidad y generalización.** Lake & Baroni (2018) mostraron, de forma muy influyente, que los modelos seq2seq estándar **no** generalizan sistemáticamente a composiciones nuevas, incluso viendo cada pieza por separado en entrenamiento —el mismo problema que motiva a NLI desde el inicio (recordá la Sección 1). El *Compositional Recursive Learner* (Chang et al., 2019) y métodos basados en recursión (Cai et al., 2017) intentan atacar esto con sesgos inductivos explícitos, aunque a menudo requieren supervisión fuerte o tienen escalabilidad limitada.
- **Aprendizaje de representaciones discretas.** El **VQ-VAE** (van den Oord et al., 2017) y el **Gumbel-Softmax** (Jang et al., 2017; Maddison et al., 2017) son las dos técnicas dominantes para entrenar variables discretas de punta a punta con gradientes, ya comparadas en la Sección 5.3. Como analogía en aprendizaje por refuerzo, el paper menciona el *options framework* (Sutton et al., 1999), que define acciones reutilizables y temporalmente extendidas que se componen en comportamientos complejos —conceptualmente muy cercano a lo que hacen los tokens del codebook de NLI, pero en el dominio del control en vez del de programas.

¿Por qué importa esto?

Fijate que casi todos los “ingredientes” de NLI —búsqueda en espacio latente (de LPN), representaciones discretas reutilizables (de CompILE y el Latent Programmer), y relajaciones diferenciables de discreción (de VQ-VAE/Gumbel-Softmax)— ya existían por separado en la literatura. La contribución de NLI no es inventar una técnica completamente nueva desde cero, sino **combinar** estas piezas de una forma específica (ejecución recurrente token a token + búsqueda por gradiente en la relajación discreta) que resulta en una propiedad emergente —la generalización composicional OOD— que ninguna de las piezas por separado lograba. Vale la pena notar esto como patrón general en investigación: muchas contribuciones importantes son *recombinaciones novedosas* de ideas existentes, no solo invenciones desde cero.

Para pensar

De las cinco familias descritas arriba, ¿cuál te parece la más cercana en espíritu a NLI, y cuál la más lejana? Intentá justificar tu respuesta pensando específicamente en *qué* generaliza cada familia y *cómo* lo logra (supervisión fuerte, sesgo arquitectónico, búsqueda en tiempo de prueba, etc.).

7 Conclusión, limitaciones y trabajo futuro

En síntesis, el paper propone NLI como una nueva forma de cerrar la brecha simbólico-neuronal planteada al comienzo (Sección 1): aprender, de punta a punta y solo a partir de ejemplos, tanto un *lenguaje* discreto propio como un *intérprete* diferenciable para él, lo que habilita búsqueda por gradiente en tiempo de prueba. Los resultados muestran que NLI supera a métodos puramente neuronales comparables y es competitivo con métodos neuro-simbólicos que tienen acceso a información privilegiada (programas verdaderos), y las ablaciones confirman que *cada* pieza del diseño —discreción, ejecución recurrente, y la propia búsqueda— es causalmente necesaria para el resultado.

Es igual de importante, para tu formación como investigador o practicante, prestar atención a las **limitaciones** que los propios autores reconocen abiertamente. Un buen hábito al leer cualquier

paper es buscar siempre esta sección primero, antes incluso que los resultados:

- **Costo computacional de la búsqueda.** La búsqueda por gradiente en tiempo de prueba, con múltiples arranques y múltiples pasos (Sección 5.4), es más cara que una simple pasada hacia adelante. Aunque el paper reporta que NLI es sorprendentemente robusto incluso con presupuestos de búsqueda restringidos, escalar a problemas más difíciles probablemente requerirá estrategias de inferencia más eficientes —los autores sugieren búsqueda local o evolutiva como direcciones prometedoras.
- **Riesgo de gradientes inestables al escalar.** A medida que los programas necesarios crecen en longitud y el vocabulario crece en tamaño, existe el riesgo teórico de gradientes que exploten o se desvanezcan (un problema clásico de redes recurrentes profundas, dado que el intérprete es, en esencia, una RNN). Los autores no lo observaron en sus experimentos, pero reconocen que podría requerir modificaciones arquitectónicas a mayor escala.
- **Poder expresivo limitado del intérprete actual.** Cada paso del intérprete condiciona en *exactamente un token*, lo que impide representar primitivas *parametrizadas* (por ejemplo, `add(k)` para un k variable, en vez de tener un token distinto por cada valor posible de k). Además, la ejecución sigue un flujo estrictamente **secuencial**, sin ningún tipo de **ramificación condicional** (*if/else*), lo cual limita la clase de programas que NLI puede, en principio, representar.

Intuición: por qué esto no es un defecto, sino ciencia honesta

Que un paper reconozca sus propias limitaciones no le resta mérito; al contrario, es señal de rigor. Ningún trabajo resuelve un problema por completo de una sola vez, y entender exactamente *dónde* se detiene la contribución actual es tan valioso como entender la contribución misma: es literalmente el mapa de qué preguntas siguen abiertas para la próxima generación de investigadores (quizás vos, si seguís por este camino).

¿Por qué importa esto?

La limitación de la falta de ramificación condicional es particularmente interesante desde una perspectiva de teoría de la computación: un lenguaje sin condicionales ni recursión (solo composición secuencial de primitivas fijas) no es Turing-completo. Esto sitúa a NLI, en su forma actual, más cerca de lenguajes de flujo de datos simples que de lenguajes de programación generales —lo cual está perfectamente bien para los dominios evaluados, pero marca un límite claro de hasta dónde puede llegar sin extensiones arquitectónicas futuras.

Para pensar

Si tuvieras que extender NLI para soportar condicionales (`if condition then f(x) else g(x)`), ¿qué cambiarías en el Algoritmo 1 de la Sección 4.3? Pista: pensá en qué información necesitaría la red de ejecución d para decidir *qué rama tomar*, y en cómo podrías mantener esa decisión diferenciable (quizás usando la misma idea de interpolación suave del mecanismo de *skip-gating*).

8 Glosario de términos clave

Ablación

Experimento en el que se remueve un componente del modelo, de a uno, para medir cuán necesario

es para el resultado observado (Sección 5.2).

Codebook

El vocabulario aprendido de K tokens discretos que el inductor puede usar para representar programas (Sección 4.2).

DSL (Domain-Specific Language)

Lenguaje de programación diseñado a mano, restringido a un dominio concreto, usado tradicionalmente por métodos simbólicos de síntesis de programas (Sección 1).

Generalización composicional

Capacidad de un modelo de recombinar piezas aprendidas de forma aislada para resolver problemas nuevos, estructuralmente distintos a los vistos en entrenamiento (Sección 1).

Gumbel-Softmax

Relajación continua y diferenciable de un muestreo categórico discreto, que permite entrenar decisiones discretas con descenso de gradiente (Sección 4.2).

ID / OOD (In-Distribution / Out-of-Distribution)

ID se refiere a evaluar el modelo en datos de la misma distribución que el entrenamiento; OOD, en datos de una distribución distinta o más difícil, diseñada para probar generalización real (Sección 3).

Leave-one-out

Estrategia de entrenamiento en la que se esconde un ejemplo de la especificación al momento de inducir el programa, y se evalúa la predicción sobre ese ejemplo escondido, forzando generalización en vez de memorización (Sección 4.1).

LAN (Latent Adaptation Network)

Familia de arquitecturas encoder-decoder que aprenden un espacio latente que se *busca* (optimiza) en tiempo de prueba para adaptarse a una tarea concreta (Sección 2).

LPN (Latent Program Network)

Arquitectura LAN predecesora de NLI, que usa un espacio latente *continuo* de un único vector (Sección 2).

NLI (Neural Language Interpreter)

La arquitectura propuesta en este paper: aprende un lenguaje discreto propio y un intérprete diferenciable para ejecutarlo (todo el documento).

PBE (Programming by Example)

Paradigma en el que la única información disponible sobre un programa es un conjunto de pares entrada-salida que produce (Sección 3).

Skip-gating / token de skip

Mecanismo del intérprete que permite “saltarse” un paso de ejecución (no-op), interpolando entre el estado anterior y el candidato nuevo, lo que permite representar programas más cortos que la longitud máxima fija (Sección 4.3).

Straight-through estimator

Técnica usada en VQ-VAE para pasar gradientes a través de una operación de selección discreta no diferenciable, tratándola como si fuera la identidad durante el retropropagado (Sección 5.3).

Temperatura (annealing / recocido)

Hiperparámetro τ que controla cuán “suave” o “dura” es una distribución de probabilidad generada por Softmax; se reduce gradualmente durante el entrenamiento o la búsqueda para pasar de exploración amplia a compromiso discreto (Secciones 4.2 y 4.4).

VQ-VAE (Vector-Quantized VAE)

Arquitectura alternativa a Gumbel-Softmax para aprender representaciones discretas, basada en un codebook y el straight-through estimator (Sección 5.3).

9 Autoevaluación final

Antes de dar por cerrada tu lectura, intentá responder estas preguntas **sin volver a mirar el texto**. No hay respuestas “oficiales” al final del documento a propósito: la idea es que las contrastes con el propio paper, o que las conversemos juntos si quieres profundizar en alguna.

Para pensar1

Explicá, con tus propias palabras y sin usar la palabra “discreto”, por qué representar programas como secuencias de tokens en vez de como un único vector continuo (la diferencia entre NLI y LPN) ayuda a la generalización composicional.

Para pensar2

¿Cuáles son las dos funciones que cumple la temperatura τ en Gumbel-Softmax, una durante la fase de exploración y otra al final del entrenamiento/búsqueda? ¿Qué pasaría en cada extremo (temperatura muy alta, temperatura muy baja) si se usara de forma constante, sin recocido?

Para pensar3

En la Ecuación 1 (objetivo de entrenamiento completo), ¿qué pasaría si se eliminara por completo el término $\lambda_{\text{reg}} \cdot \mathcal{L}_{\text{reg}}$? Conectá tu respuesta con los resultados de ablación de la Sección 5.2.

Para pensar4

Describí, paso a paso y sin mirar el pseudocódigo, qué hace el mecanismo de *skip-gating* en el Algoritmo 1, y por qué es necesario para que programas de distinta longitud quepan en una secuencia de longitud fija T .

Para pensar5

¿Por qué la búsqueda en tiempo de prueba de la Sección 4.4 solo es posible porque el intérprete es diferenciable? ¿Qué tendría que cambiar en el diseño si quisiéramos “buscar” un programa para un intérprete simbólico tradicional, no diferenciable?

Para pensar6

Los resultados de la Tabla 1 muestran que *todos* los métodos logran casi 100% ID, pero solo NLI con búsqueda por gradiente logra un desempeño alto OOD. ¿Qué te dice esto sobre la utilidad (o la falta de ella) de medir solamente precisión in-distribution al evaluar un modelo de síntesis de programas?

Para pensar7

NLI logra resultados competitivos con métodos neuro-simbólicos en DeepCoder *sin* tener acceso a programas verdaderos durante el entrenamiento. ¿En qué escenarios del mundo real esto sería una ventaja decisiva? ¿Podés pensar en un dominio donde, en cambio, sí sería razonable y barato conseguir esa supervisión adicional?

Para pensar8

De las limitaciones reconocidas en la Sección 7 (costo de búsqueda, riesgo de gradientes inestables, falta de primitivas parametrizadas y de ramificación condicional), ¿cuál te parece la más fácil de resolver con una extensión incremental del diseño actual, y cuál te parece que requeriría repensar la arquitectura desde una base distinta?

Para pensar9 — integradora

Si tuvieras que explicarle todo el paper a alguien en 3 frases, sin ecuaciones, ¿qué dirías? Intentalo por escrito antes de seguir leyendo cualquier otro material.

Para pensar10 — para ir más allá

¿A qué otro problema de tu interés (no necesariamente síntesis de programas) le aplicarías el patrón general de NLI —aprender una representación discreta y componible, junto con un ejecutor diferenciable de esa representación, para luego poder *buscar* soluciones con gradiente en tiempo de inferencia? Intentá justificar por qué ese problema podría beneficiarse de esta idea.